

DAY 2 · MODULE 3 · 25 MIN

# Evaluating Retrieval

Know if your knowledge layer actually works

# Why evaluate retrieval separately

If you skip this step you'll do one of two things:

- Ship an agent that seems to work — until it doesn't, and they can't tell why
- Tune prompts forever, when the real problem was chunking or embeddings

***Retrieval failures and generation failures look identical from the outside. Evaluate separately, fix separately.***

*Day 4 anchors evaluation for full workflows. Today's module gets the habit started.*

# Two evaluation modes for RAG

| Mode               | What it evaluates                                         | Ground truth required |
|--------------------|-----------------------------------------------------------|-----------------------|
| Process evaluation | The retrieval step itself — did we get relevant chunks?   | Depends on evaluator  |
| System evaluation  | The end-to-end response — did the agent answer correctly? | Sometimes             |

- Process eval finds the retrieval-side bug
- System eval confirms the whole thing works
- You'll do a small dose of each in today's lab

# The Foundry evaluator catalog for RAG

| Evaluator                       | Type    | Needs ground truth?                                |
|---------------------------------|---------|----------------------------------------------------|
| Retrieval                       | Process | No — LLM judges context relevance                  |
| Groundedness                    | System  | No — LLM judges if response is grounded in context |
| Groundedness Pro (preview)      | System  | No — Content Safety service, boolean               |
| Relevance                       | System  | No — LLM judges if response addresses query        |
| Response Completeness (preview) | System  | Yes — needs expected answer                        |
| Document Retrieval              | Process | Yes — needs query-relevance labels (qrels)         |

*Start with the top four. Add ground truth when you're ready to invest in labels.*

# Where to start — Retrieval and Groundedness

*Two zero-setup RAG evaluators — one process, one system. Neither needs ground truth.*

## Retrieval (process)

- Input: query, retrieved context
- Output: 1–5 score (pass  $\geq 3$ ), plus reasoning
- Answers: 'Are the chunks we pulled actually relevant to the question?'

## Groundedness (system)

- Input: query, response, retrieved context
- Output: 1–5 score (pass  $\geq 3$ ), plus reasoning
- Answers: 'Did the response stay in the context, or did the model fabricate?'

*Together, these two isolate retrieval-side vs. generation-side failure.*

# Groundedness vs. Response Completeness

*Two sides of the same coin:*

- Groundedness = precision. Did the response contain anything not in the context? (Fabrication check.)
- Response Completeness = recall. Did the response leave anything out that the ground truth expected? (Coverage check.)

A high-groundedness, low-completeness response is accurate but partial. A low-groundedness, high-completeness response is comprehensive but making things up. Track both.

*Response Completeness needs ground truth — invest when you have a benchmark corpus.*

# Document Retrieval — the deep debug tool

*When retrieval quality is the bottleneck, this is your parameter-sweep evaluator.*

- Input: query-relevance labels (qrels) and the ranked retrieval output
- Output: NDCG, XDCG, Fidelity, Max Relevance, Holes at various k
- Use it to answer:
  - Should I use vector, keyword, or hybrid?
  - What's the right top\_k?
  - Is 500-token chunking better than 1000?

*You need labeled data — a person judged which docs are relevant for each query. Worth it when tuning matters.*

# Configuring an evaluator (Python SDK)

```
testing_criteria = [  
    {  
        "type": "azure_ai_evaluator",  
        "name": "groundedness",  
        "evaluator_name": "builtin.groundedness",  
        "initialization_parameters": {"deployment_name": model_deployment},  
        "data_mapping": {  
            "context": "{{item.context}}",  
            "response": "{{item.response}}",  
        },  
    },  
    {  
        "type": "azure_ai_evaluator",  
        "name": "retrieval",  
        "evaluator_name": "builtin.retrieval",  
        "initialization_parameters": {"deployment_name": model_deployment},  
        "data_mapping": {"query": "{{item.query}}", "context": "{{item.context}}"},  
    },  
]
```

*Same JSON shape for each evaluator. The data\_mapping tells the evaluator where to find fields on your test dataset.*



# The test dataset — smaller than you think

A JSONL file, one line per test case:

```
{ "query": "How do I set up a Prompt agent?",  
  "context": "A Prompt agent is authored in the Foundry portal or SDK...",  
  "response": "You create a Prompt agent by..." }  
{ "query": "What models does Foundry support?",  
  "context": "Foundry hosts models from Azure OpenAI, Anthropic, Meta...",  
  "response": "Foundry supports multiple models including..." }
```

- Start with 10–15 examples
- Enough signal to catch obvious regressions; small enough to actually maintain
- Add examples over time as you find failure modes

# Reading the output

```
{
  "type": "azure_ai_evaluator",
  "name": "Groundedness",
  "metric": "groundedness",
  "score": 4,
  "label": "pass",
  "reason": "The response is well-grounded without fabricating content.",
  "threshold": 3,
  "passed": true
}
```

- score — 1–5 numeric (or true/false for Pro)
- label / passed — pass or fail against the threshold
- reason — the LLM judge's explanation (read this when things fail)
- metric — which evaluator produced this row

***The reason field is where debugging happens. Read it.***

# The iteration loop

*Same shape as Day 1's prompt-engineering loop:*

- Author an eval dataset (10–15 items)
- Run Retrieval + Groundedness against your current pipeline
- Read the failures — retrieval bug or generation bug?
- Change one thing (chunk size, top\_k, prompt, model)
- Rerun. Compare scores.

***Change one thing at a time. Three changes at once = you learned nothing about what worked.***

*Same discipline shows up Day 4 for multi-agent workflow eval.*

# LLM-as-judge caveats

You're using an LLM to judge an LLM. Some caveats:

- Judge bias — the judge model has preferences. Same prompt, different judges = different scores.
- Judge cost — every eval item is an extra LLM call. Budget accordingly.
- Judge drift — model updates can shift baseline scores. Version-pin your judge.
- Judge as a check, not truth — cross-check with the reason field and spot-check the actual failures.

*Day 5 covers online eval and drift more deeply. Today: know the judge isn't infallible.*

# Common traps

- Testing end-to-end only — a bad answer might be retrieval OR generation; can't tell without process eval
- Test set too small — 3 items = coin flip; 10–15 is the useful floor
- Test set too large — every eval run costs LLM calls; 50 hand-labeled > 5,000 unlabeled
- Changing three things at once — you learn nothing about what worked
- Ignoring the reason field — score tells you if it failed; reason tells you why
- Judge model = production model — conflict of interest; use a different (usually smaller) judge

# Where eval shows up the rest of the week

- Day 3 (single-agent depth) — evaluator patterns extend to tool-use correctness
- Day 4 (multi-agent anchor) — trajectory eval, cost-per-successful-outcome, regression harness
- Day 5 (production) — online eval, drift detection, red-teaming
- Capstone — required elements include a golden set + eval scores

*The habit that starts today runs through the whole week.*

# Takeaways

- Evaluate retrieval separately or you can't tell retrieval bugs from generation bugs.
- Foundry evaluators: start with Retrieval (process) + Groundedness (system) — both zero-setup.
- Groundedness (precision) + Response Completeness (recall) — track both when you have ground truth.
- Small test sets beat big untested claims — 10–15 hand-labeled items is the floor.
- Change one thing, then rerun — same iteration loop as prompt engineering.

**Next:** Tools layer — how agents do things beyond talking.